JÖNKÖPING UNIVERSITY
School of Engineering

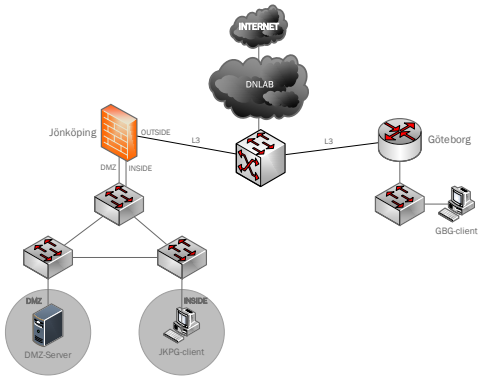
IT-arkitektur och IT Service Management

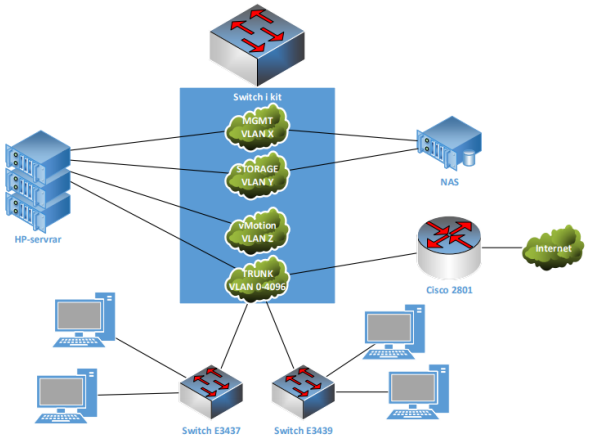
Infrastrukturarkitektur

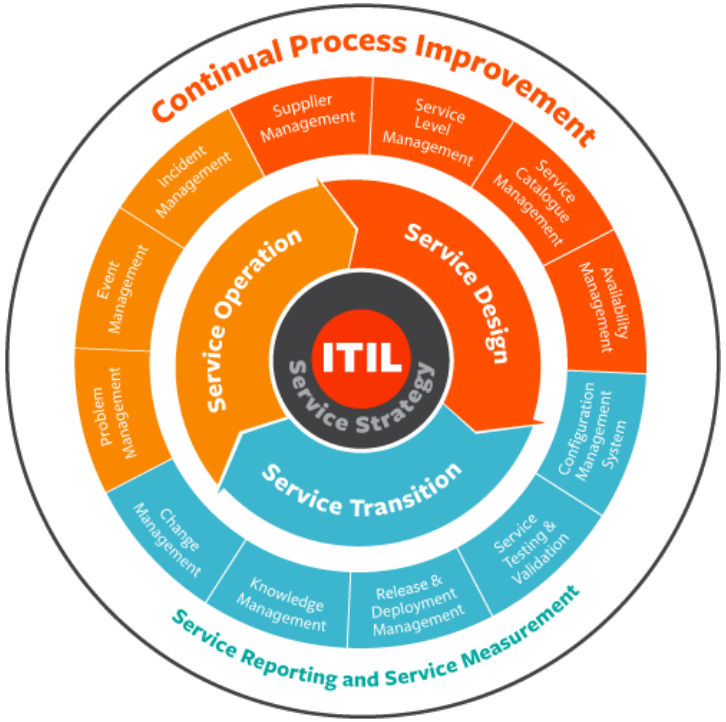
Planering

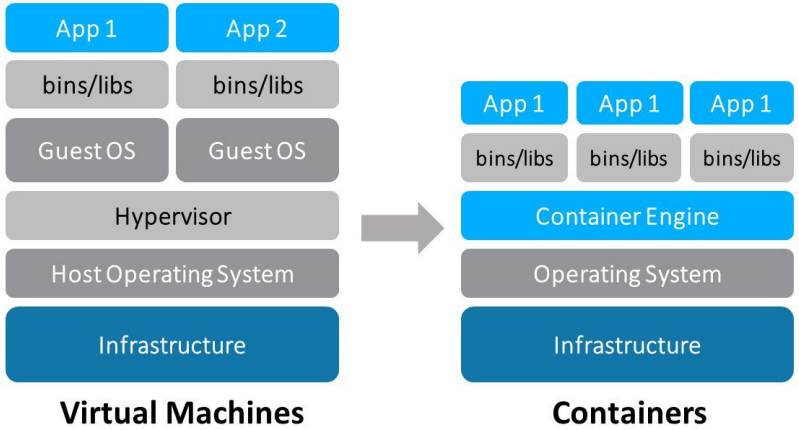
Vecka	Dag	Moment	Lärare
44	Tis	~~**Föreläsning** – Kursintroduktion & arkitektroller (kapitel 1, IASA-papper)~~	Sonny
	Fre	~~**Föreläsning** – Enterprisearkitektur (kapitel 2)~~	Ulf
	Sön	~~**Deadline Uppgift 1** – Arkitektroller~~	
45	Mån	~~**Föreläsning** – Verksamhetsarkitektur (kapitel 3)~~	Ulf
	-	~~**Föreläsning** – Modellering (inspelad)~~	Sonny
	Ons	~~**Karriärum & Handledning** – Uppgift 2~~	Sonny
	Sön	~~**Deadline Uppgift 2** – Modellering i ArchiMate~~	
46	Mån	~~**Föreläsning** – Lösnings- och mjukvaruarkitektur (kapitel 4-5)~~	Sonny
	Tis	**Föreläsning** – Infrastrukturarkitektur (forts. kapitel 4)	Sonny
	Sön	**Deadline Uppgift 3** – Rollen som infrastrukturarkitekt	
47	Ons	**Föreläsning** – ITIL del 1	Hannes
	Ons	**Seminarium 1 (ITSM-projektet)**	Hannes
48	Tor	**Föreläsning** – ITIL del 2	Hannes
	Tor	**Seminarium 2 (ITSM-projektet)**	Hannes
49	Ons	**Föreläsning** – ITIL del 3	Hannes
	Ons	**Seminarium 3 (ITSM-projektet)**	Hannes
50	Lör	**Tentamen**	
	-	Extra inlämning **Uppgift 1-3** **& Deadline** **ITSM-projektet**	

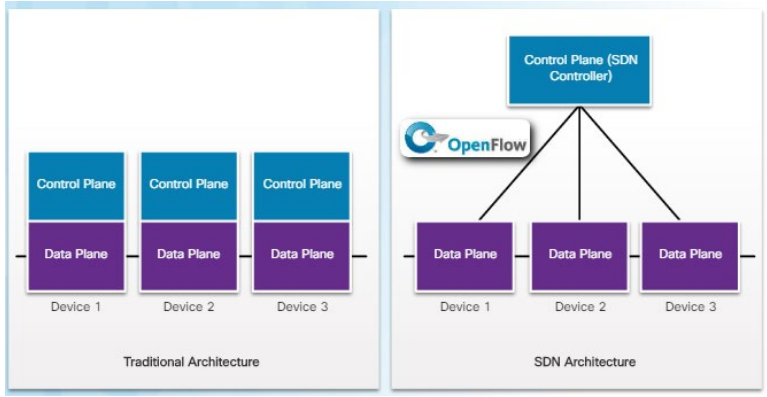
Hur har vi pratat om infrastrukturarkitektur tidigare?



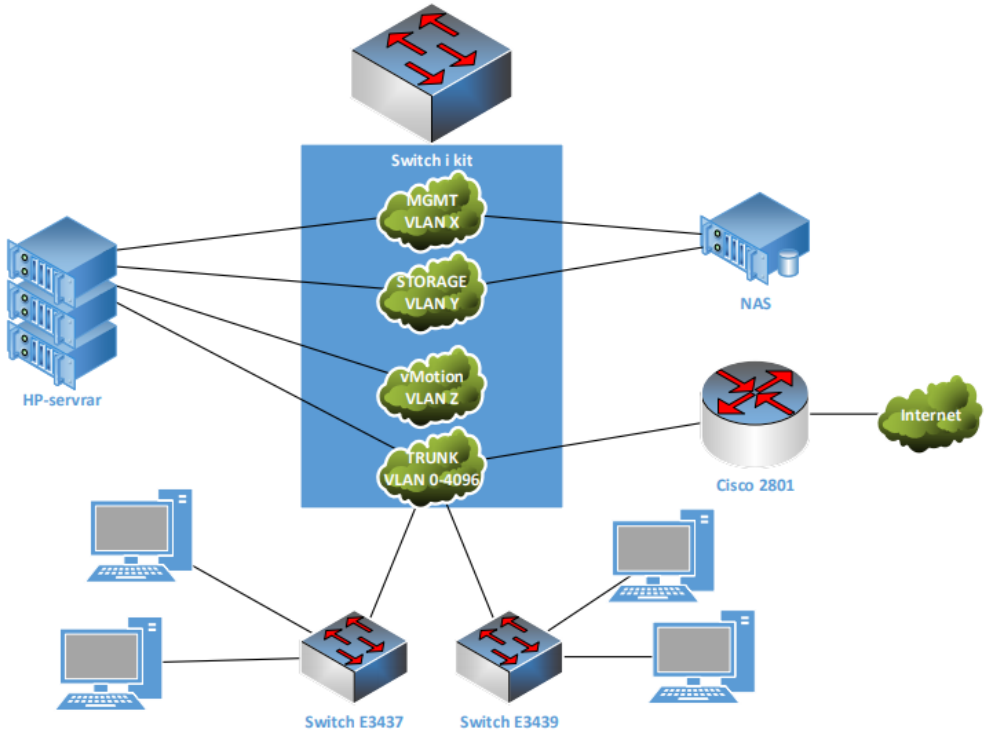


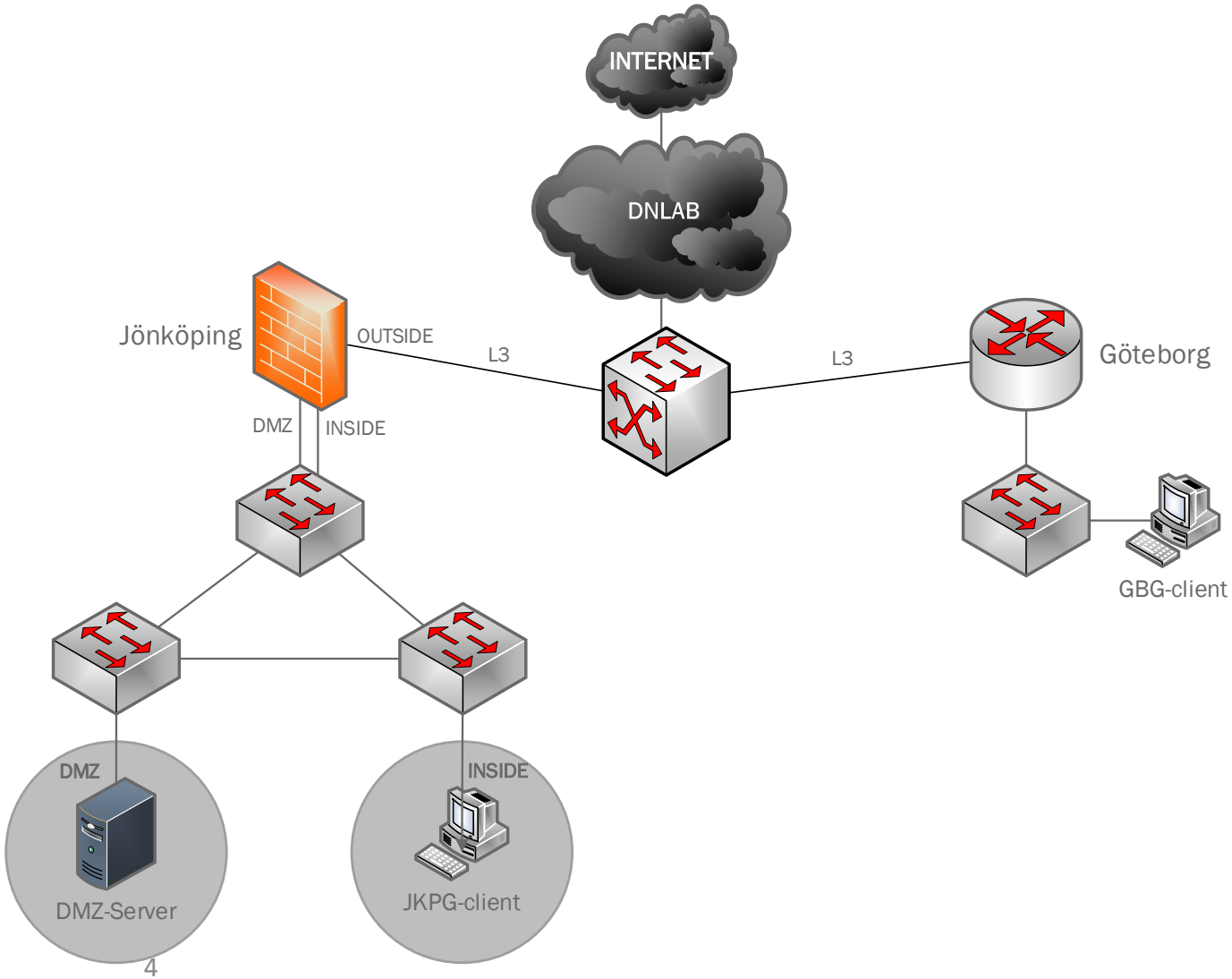




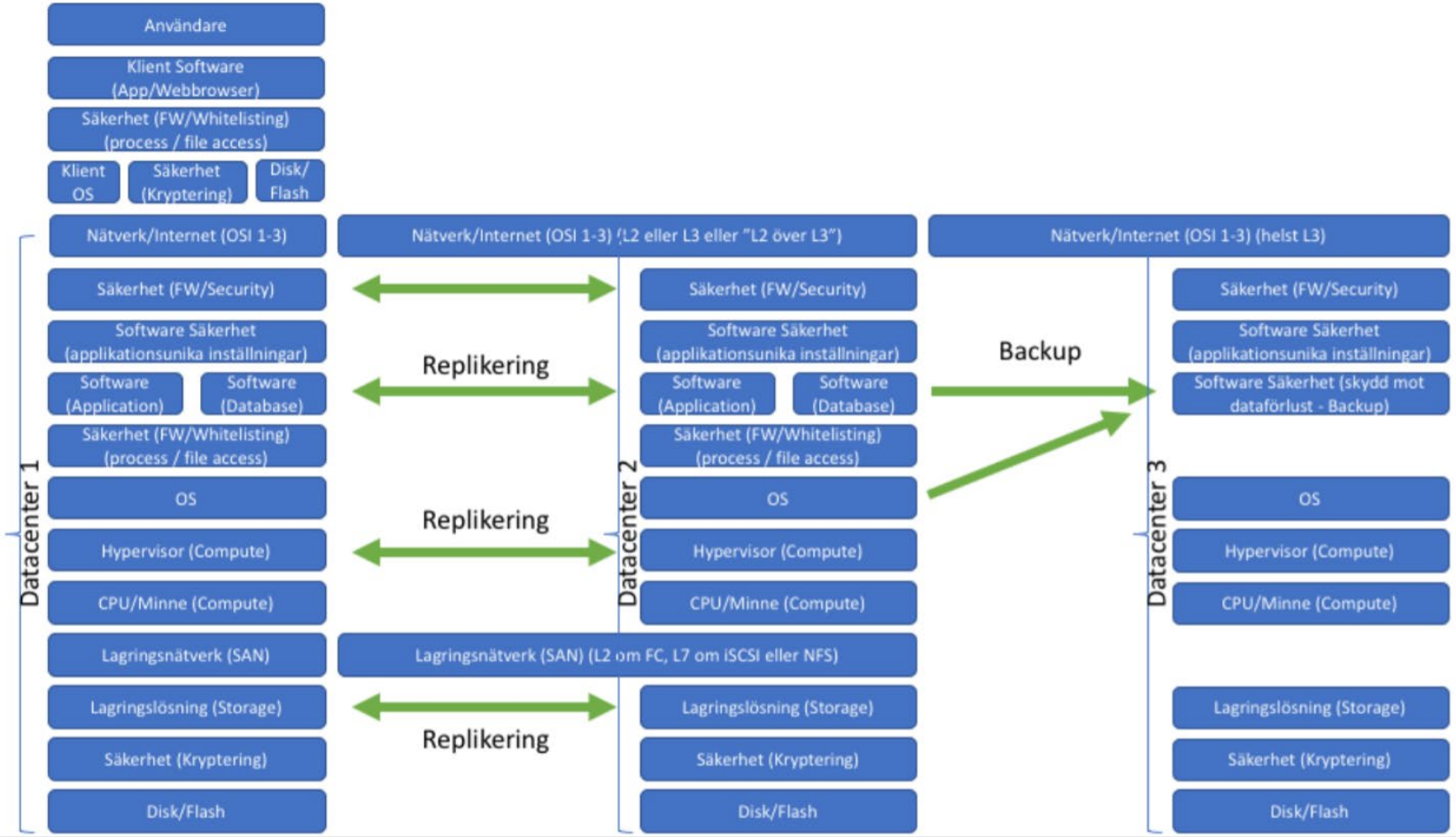


Hur har vi modellerat?





Hur har vi modellerat?



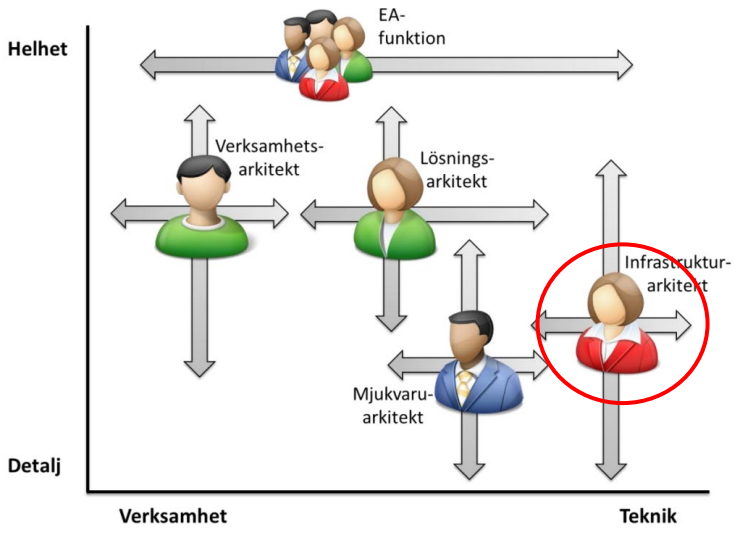
Kapitel 4

Lösningsarkitektur (forts.)

Var avslutade vi förra föreläsningen?

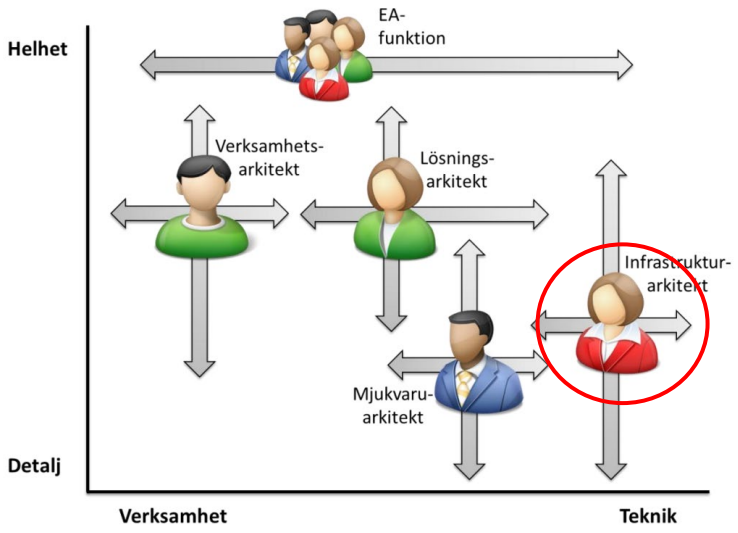
- Själva utvecklingen av systemen och framtagandet av dess integrationer är av engångskaraktär, längst befinner sig dock mjukvaran i **förvaltningsfasen** (största delen av kostnaden)
- En stor del av förvaltningsfasen är att hantera den dagliga driften och användarsupport (IT Ops)
- Mycket av arbetet här är det ni har lärt er i allt från tekniska kurser
  - En föreläsning för mjukvaruutvecklare hade kunna ha ett stort fokus på detta

Infrastrukturarkitekten 1 (2)



- Nära infrastrukturen som stödjer mjukvaran
- Ansvarig för rätt infrastruktur i form av:
  - Applikationsstöd och tjänster i servermiljö
  - Kommunikation/nätverksmiljö
  - Lagring och lagringssystem
  - IT-säkerhet
- Mycket arbete med löpande drift, men man bör ändå involvera infrastrukturarkitekten tidigt i projekt
- Överlämning, driftsättning, drift och övervakning ofta via **ITIL**

Infrastrukturarkitekten 2 (2)



- Kapacitetshantering (**capacity management**) och tillgänglighetshantering (**availability management**)
  - Modeller för skalning, lastbalansering, HA
- Bedömer driftsalternativ, in-house, moln, etc.
- **Leverabler:** topologidiagram, övervakning, tekniska riktlinjer för identitetshantering, servrar, databaser, etc.
- God kompetens inom infrastruktur, och ofta ett team av system- och nätverkstekniker som stöd

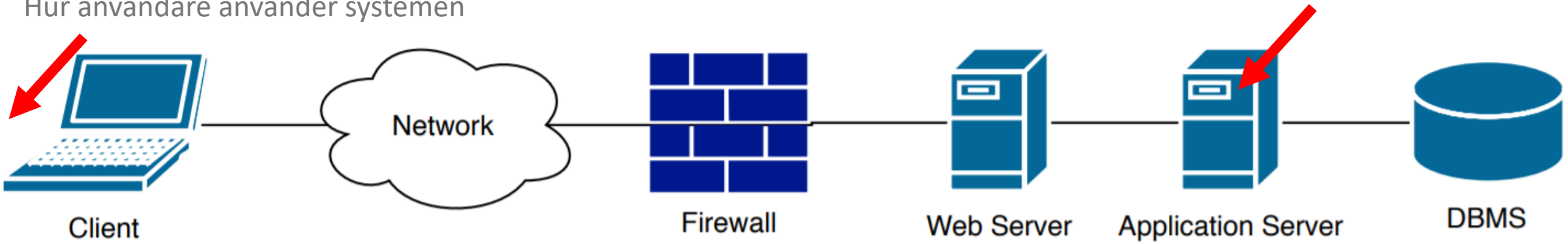
Begreppsförvirring

- Infrastrukturarkitekten kan även vara en ”lösningsarkitekt” för infrastruktur/cloud/nätverk/säkerhet

Infrastrukturarkitektens ansvar

Verksamhetsarkitektens ansvar:
Hur användare använder systemen

Mjukvaruarkitektens ansvar:
Mjukvaran som körs på applikationsservern



Support
Operativsystem
Applikationsdistribution
Nätverksåtkomst
Inköp
Licenshantering
Utbildning av användare
MDM
Backuper

Internetåtkomst
Tillgänglighet
Kapacitet
Redundans
QoS
Multi-homing
VPN
Säkerhetsåtgärder

Säkerhetsfunktioner
Åtkomstkontroll
IDS/IPS
URL-filter
Loggning
Juridiska aspekter
Övervakning
Penetrationstestning
Molnkopplingar

Lastbalansering
Åtkomst och integrationer
Virtualisering
Lagring
Koppling mot molntjänster
Säkerhetsskydd
Inköp
Licenser
Hantering av datacenter

Backuper
Lagring
Optimeringar
Felsökning
Inköp
Licenser
Replikeringar
Prestanda
SQL-optimeringar

Huvudfokus: Tillförlitlig och tillgänglig drift

- Den *teknologi*, de *processer* och den *organisation* som krävs för att enligt serviceavtal från datacentret leverera de IT-tjänster som är avtalat med kund
- Ett viktigt mål är (ofta) att designa infrastrukturen för hög tillgänglighet
- Vad är bästa sättet att säkerställa detta?
  - Simulera olika belastningar, och testa systemet och infrastrukturen!
  - **Case:** O-Ringen

MAXI
ICA STORMARKNAD
TORK
SCA
TEAM SPORTIA
SIA

Basteknologier i ett datacenter

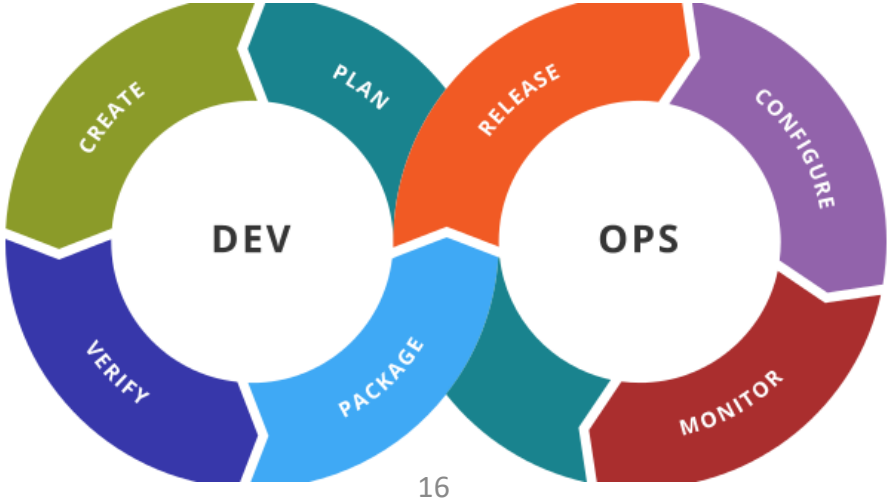
- Boken fokuserar mycket på:
  - Övervakning av den fysiska lokalen: Tillträde, temperatur, ström, brand, luftfuktighet, översvämning, seismisk aktivitet
  - Anläggningsregister (asset management) och rackstandarder
  - Strömförsörjning, kylning och släckningssystem

Teknisk infrastruktur i ett datacenter

- Det ni har lärt er om (eller ska lära er) gällande:
  - Nätverksanslutningar
  - Serverdrift
  - Virtualisering
  - Lagring
  - Applikationsdrift
  - Databaser
  - Övervakningssystem (NMS)
  - Fjärradministration

DevOps

- Som vi pratade om förra veckan:
- Traditionellt har mjukvara utvecklats (**Dev**elopment, **Dev**) och sedan levererats, förvaltats och underhållits (**IT Op**eration**s**, **IT Ops**)
- De agila arbetsmetoderna har gjort att Dev och Ops måste arbeta tätare, lösningen: **DevOps**



https://www.frontit.se/inspiration-kunskap/artiklar/devops-ci-cd-vad-betyder-det-egentligen/

Rollen för infrastruktur (IT Ops) i DevOps-miljöer

- Säkerhetsfokuset blir allt viktigare, SOC-arbete
- Basinfrastrukturen finns fortfarande kvar, DNS, DHCP, AD, backup, etc.
- Minskad tid för driftsättning och servicefönster
  - Ökad tid för att förbereda automatiserad driftsättning utan servicefönster
- Övervakning och logghantering blir mer komplext (många containrar)
- Större fokus på molntjänster (privata, publika, hybrida)
- Löpande administration och förvaltning, nya metoder för DevOps:
  - Configuration Management (CM), t.ex. via Puppet och Ansible
  - Orkestrering, t.ex. via Kubernetes

Drifts- och förvaltningsprocesser

- I mångt och mycket enligt ITIL:
  - **Identitetshantering**
  - **Kapacitetshantering**
  - **Konfigurationshantering**
  - **Val av driftsplattform**
  - Säkerhetsarkitektur
  - Releasehantering
  - Förändringshantering
  - Incidenthantering
  - Problemhantering
  - Kontinuitetshantering
  - Klienthantering

Identitetshantering

Identity Management (IdM, IAM, IdAM)

Identitetshantering (IdM)

- En viktig del för en infrastrukturarkitekt är att bestämma principer för **identitetshantering**
- Identitetshantering är en delmängd av informationssäkerhet
  - Har ni haft kurser om, och har vi pratat om under flera utav de tidigare kurserna!
- När företag flyttar till molnet blir detta mer komplext med identitetskopplingar mot olika system
- Utan central identitetshantering krävs att varje tjänst (t.ex. Canvas, Ladok, mail) har ett eget system för lösenord
- Ni har arbetat tidigare med identitetshantering i **Active Directory**

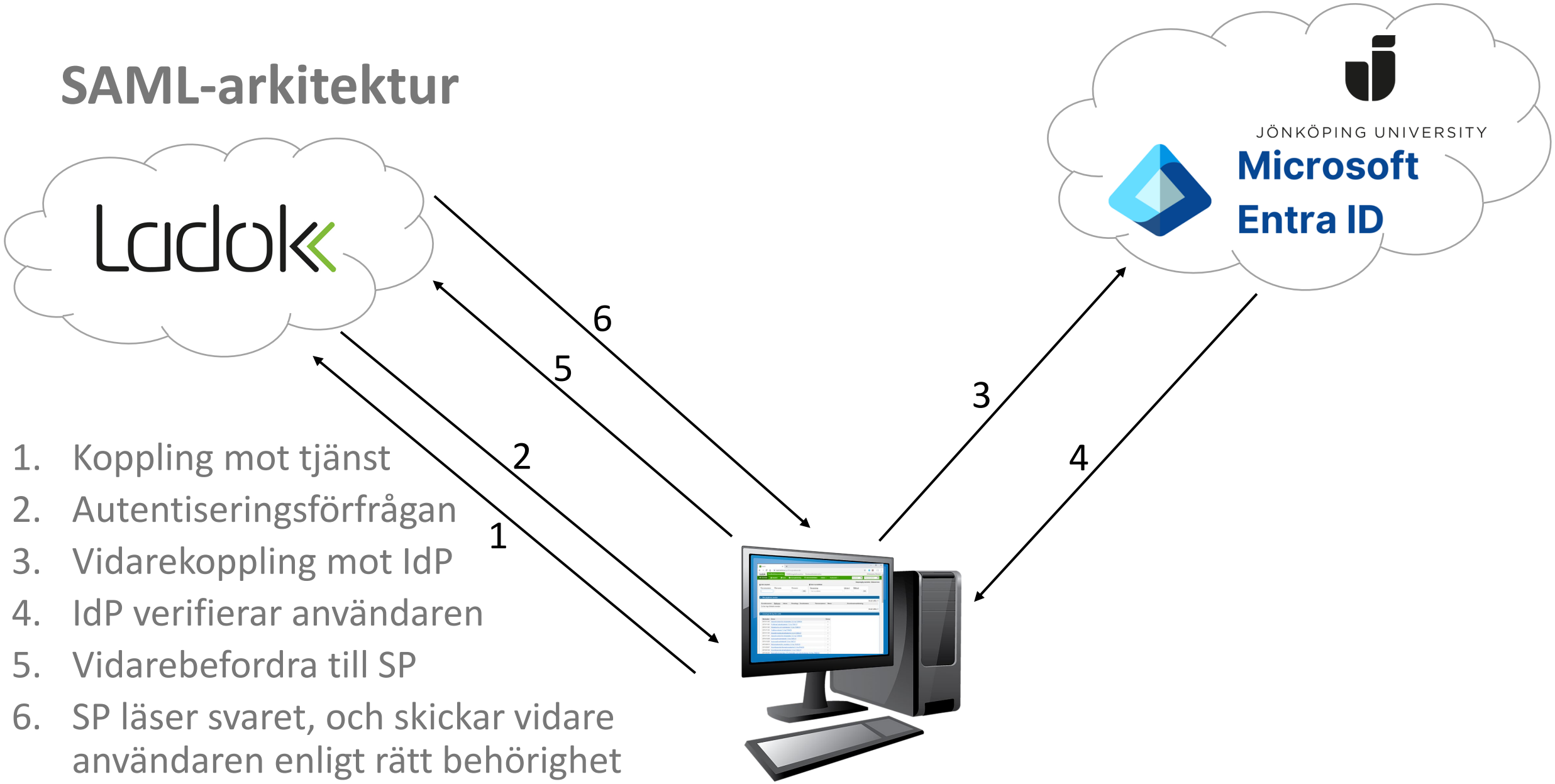
Federation

- **Federation** eller **Federated Identity Management** (FIdM) är ett koncept där identiteter kan delas mellan olika domäner och kan bland annat ge funktioner för **Single-Sign On (SSO)**
- Autentisering hanteras via en **IdP (Identity Provider)** hos en organisation som verifierar identiteten och krav på exempelvis:
  - Komplexitet på lösenordet
  - Glömt lösenord och byte av lösenord
  - Multifaktor-autentisering (MFA)
- För federation används protokoll som SAML, OAuth, OpenID samt tjänster som AD FS

SAML

- För att federation ska fungera mellan två parter måste vi **följa standarder** och lita på **varandra**
- **SAML (Security Assertion Markup Language)** är en XML-baserad standard för kommunikation av identiteter
- Tre delar bygger upp kommunikationen i SAML:
  - **Subject:** Användare (eller system) som vill ha åtkomst till ett system
  - **Service Provider (SP):** Systemet som behöver få tag i identiteter från…
  - **Identity Provider (IdP):** Systemet som kan verifiera användarens identitet

SAML-arkitektur



1. Koppling mot tjänst
2. Autentiseringsförfrågan
3. Vidarekoppling mot IdP
4. IdP verifierar användaren
5. Vidarebefordra till SP
6. SP läser svaret, och skickar vidare användaren enligt rätt behörighet

Realm discovery

- Realm discovery – Hur vet Service Provider (SP) vilken Identity Provider (IdP) som ska kontaktas?
- Tre alternativ:
  - I vissa fall är tjänsten endast kopplad mot **en IdP** (se Canvas)
  - **Domännamn** används av användaren (se Office 365)
  - Användaren får **välja IdP** som ska användas (se Ladok)

Kapacitetshantering

Capacity Management

Kapacitetshantering

- Kapacitetshantering handlar om två saker:
  - Inte för lite kapacitet vid belastningstoppar
  - Inte för mycket kapacitet vid belastningsdalar
- En stor del handlar givetvis om **lastbalansering**, och att skala upp kapaciteten vid behov, t.ex.:
  - Vid produktsläpp
  - Black Friday
- En del handlar om att skala ner när belastningen är låg (t.ex. AWS Auto Scaling)

Baserat på: Limoncelli et al. (2014). *The Practice of Cloud System Administration*

Kapacitet: On-prem vs. Cloud

- I en miljö som är on-prem (in-house), måste man planera för infrastruktur som klarar belastningstopparna
- I en molnmiljö går det i större grad skifta kapaciteten beroende på belastning
  - Det är här vi fullt ut kan arbeta med kapacitetshantering för toppar och dalar
- För att få en automatiserad *kapacitetshantering* krävs också en automatiserad *konfigurationshantering*

Baserat på: Limoncelli et al. (2014). *The Practice of Cloud System Administration*

Konfigurationshantering

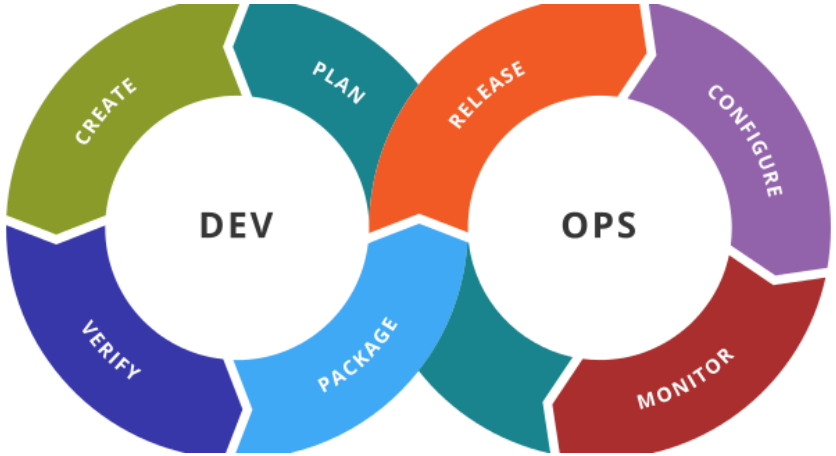
Configuration Management

Konfigurationshantering (Configuration Management)

- Består av flera **Configuration Items (CIs)**
  - Vilken konfiguration har enheter, tjänster och system?
- **Vanlig setup:** Excel-dokument som uppdateras löpande (men som ibland glöms bort)
- **Bättre metod:** Använd ett CMDB (Configuration Management Database)
  - **Demo:** iTop (admin-gränssnitt) som bygger på ITIL
- Senaste varianten är CM-verktyg som också kan styra enheter och tjänster som brukar benämnas **Infrastructure as Code (IaC)**

Infrastructure as Code (IaC)

- **Infrastructure as Code (IaC)** innebär att istället för att manuellt konfigurera infrastruktur, definieras den med kod
- Blir allt viktigare när utvecklare blir mer agila och arbetar mer mot DevOps och kontinuerlig leverans av mjukvara
- Precis som att UML kan generera delar av koden i ett system, kan en bra Ansible-playbook generera rätt konfiguration



Delar i IaC-verktyg

- Öppen eller stängd källkod
- **Kommunikationsarkitektur**
- **Typ av verktyg**
- **”Mutable” eller ”Immutable”**
- **Språk**

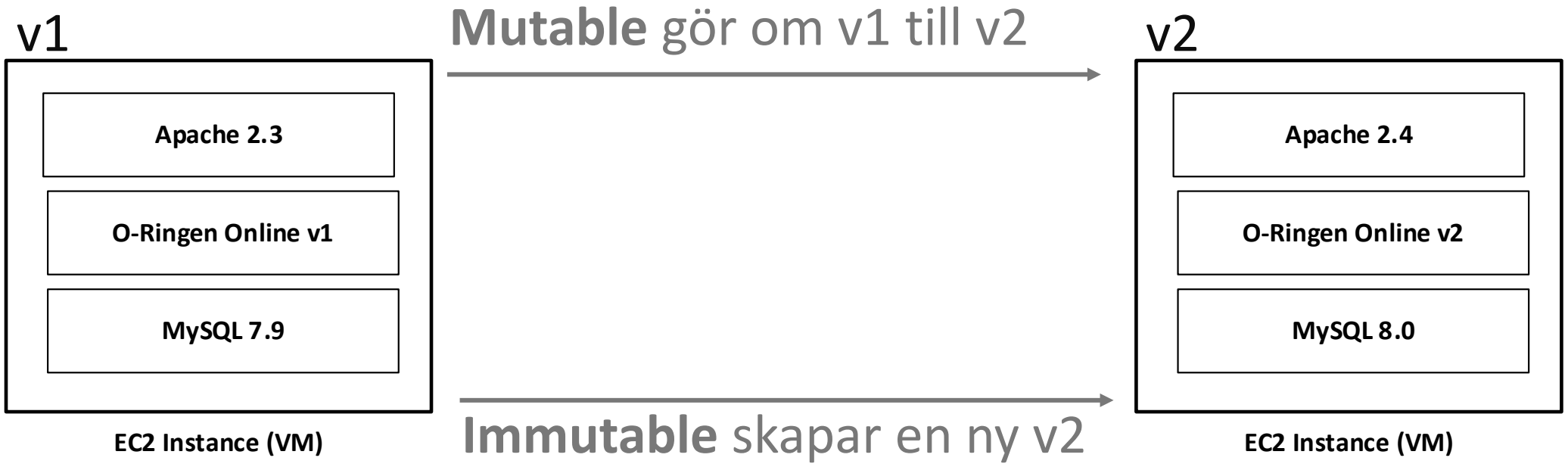
Infrastructure as Code – Kommunikationsarkitektur

- **Metod** för att få konfigurationen till enheten
  - **Push** – Skickar ut konfigurationen till enheten/systemet
  - **Pull** – Enheten/systemet hämtar själv konfigurationen
- **Agent**
  - Bygger på en agent som installeras/distribueras till enheter
  - Kallas även *Client/Server*
- **Agentlös**
  - Bygger på befintliga protokoll som används för att styra enheten, t.ex. SSH
  - Kallas även *Client-Only*

Infrastructure as Code – Typ av verktyg

- **Konfigurationshantering** – Hantera konfiguration på enheter
  - Ansible
  - SaltStack
  - Puppet
  - Chef
- **Provisionering/orkestrering** – Provisionera infrastruktur på olika plattformar
  - Terraform
  - Cloudformation
- I många fall liknande funktion, men finns miljöer där båda används, i stort handlar det om…

Infrastructure as Code – Mutable vs Immutable Infrastructure



Språk

- **Språket** hanterar hur konfiguration definieras
  - **Procedurellt** – Definiera vad som ska göras, steg för steg
  - **Deklarativt** – Definiera önskat målläge, verktyget gör i egen ordning
- Säg att ni vill skapa 5 server i AWS med Ansible (procedurellt) jämfört med Terraform (deklarativt)

Ansible (Pro.)

```
- ec2:
    count: 5
    image: ami-v1
    instance_type: t2.micro
```

Terraform (Dek.)

```
resource "aws_instance" "example" {
    count = 5
    ami = "ami-v1"
    instance_type = "t2.micro"
}
```

Procedurellt vs Deklarativt

- Vi vill nu skapa 5 servrar till…

Ansible (Pro.)

```
- ec2:
    count: 5
    image: ami-v1
    instance_type: t2.micro
```

Vad vill vi göra?

Terraform (Dek.)

```
resource "aws_instance" "example" {
    count = 10
    ami = "ami-v1"
    instance_type = "t2.micro"
}
```

Hur vill vi ha det?

Jämförelse mellan verktyg

	CHEF	puppet	ANSIBLE	SALTSTACK	CloudFormation	Terraform
Code	Open source	Open source	Open source	Open source	Closed source	Open source
Cloud	All	All	All	All	AWS only	All
Type	Config Mgmt	Config Mgmt	Config Mgmt	Config Mgmt	Orchestration	Orchestration
Infrastructure	Mutable	Mutable	Mutable	Mutable	Immutable	Immutable
Language	Procedural	Declarative	Procedural	Declarative	Declarative	Declarative
Architecture	Client/Server	Client/Server	Client-Only	Client/Server	Client-Only	Client-Only

Ansible

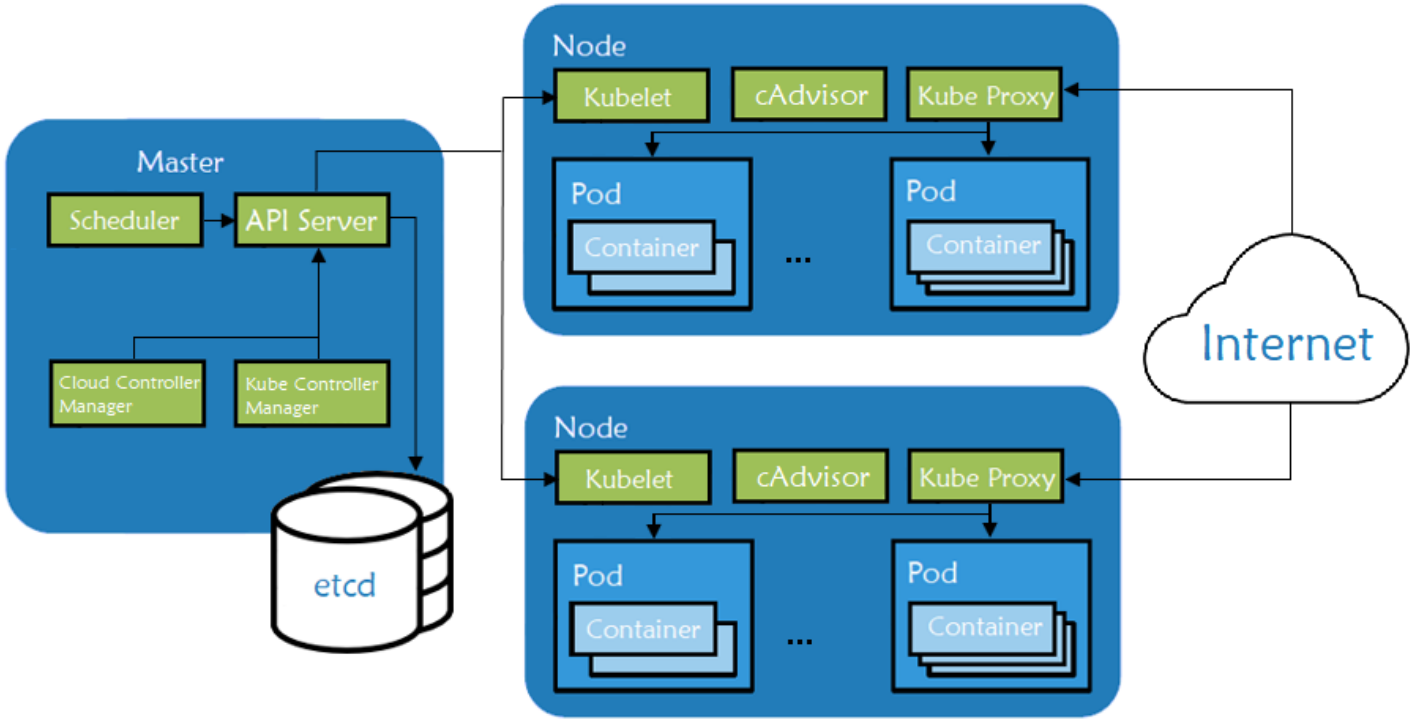
- **Ansible** är ett av de populäraste verktygen för Configuration Management
  - Öppen källkod (ägs av Red Hat)
  - Har brett stöd bland olika plattformar, men körs från Linux/Unix
  - Kör agentlöst, via t.ex. SSH och PowerShell
- Använder **playbooks** för att automatisera konfiguration och driftsättning av enheter
- Playbooks skrivs i **YAML (YAML Ain't Markup Language)**, med filändelse .yaml eller .yml

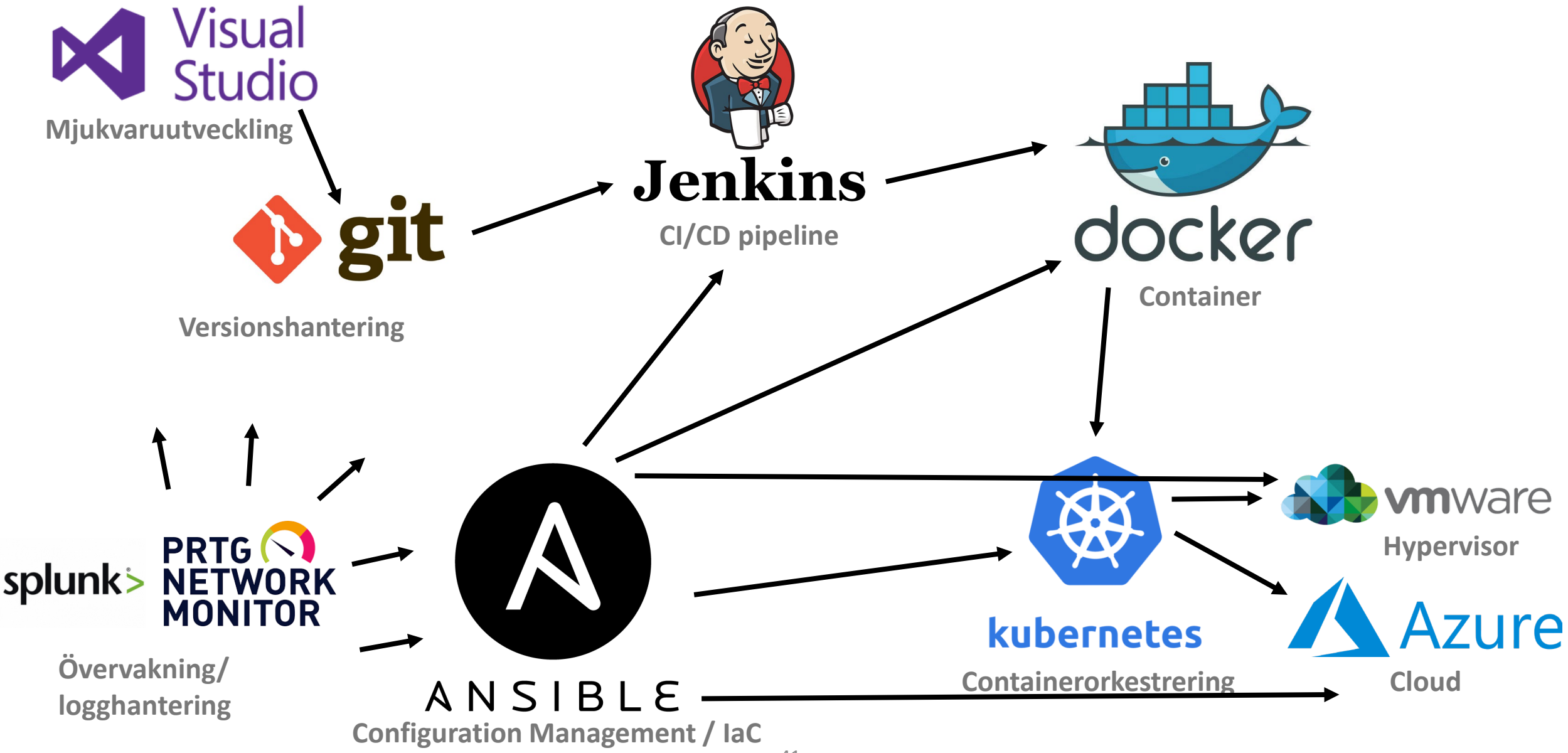
Ansible playbook (YAML)

```yaml
- name: Create local group to contain new users
  win_group:
    name: LocalGroup
    description: Allow access to C:\Development folder

- name: Create local user
  win_user:
    name: '{{ item.name }}'
    password: '{{ item.password }}'
    groups: LocalGroup
    update_password: false
    password_never_expires: true
  loop:
  - name: User1
    password: Password1
  - name: User2
    password: Password2

- name: Create Development folder
  win_file:
    path: C:\Development
    state: directory

- name: Set ACL of Development folder
  win_acl:
    path: C:\Development
    rights: FullControl
    state: present
    type: allow
    user: LocalGroup

- name: Remove parent inheritance of Development folder
  win_acl_inheritance:
    path: C:\Development
    reorganize: true
    state: absent
```

Tätt sammankopplat till CM: Kubernetes

- **Kubernetes** är ett orkestreringsverktyg för containers
- Bygger på:
  - Master (styrenhet)
  - Node (ofta en VM)
  - Pod
  - Containers
- Hur hänger allt ihop med…
  - IaC?
  - Kubernetes?
  - DevOps?
  - CI/CD?





https://thenewstack.io/case-study-efficient-infrastructure-with-containerized-pipelines-kubernetes-and-git/

Val av driftsplattform

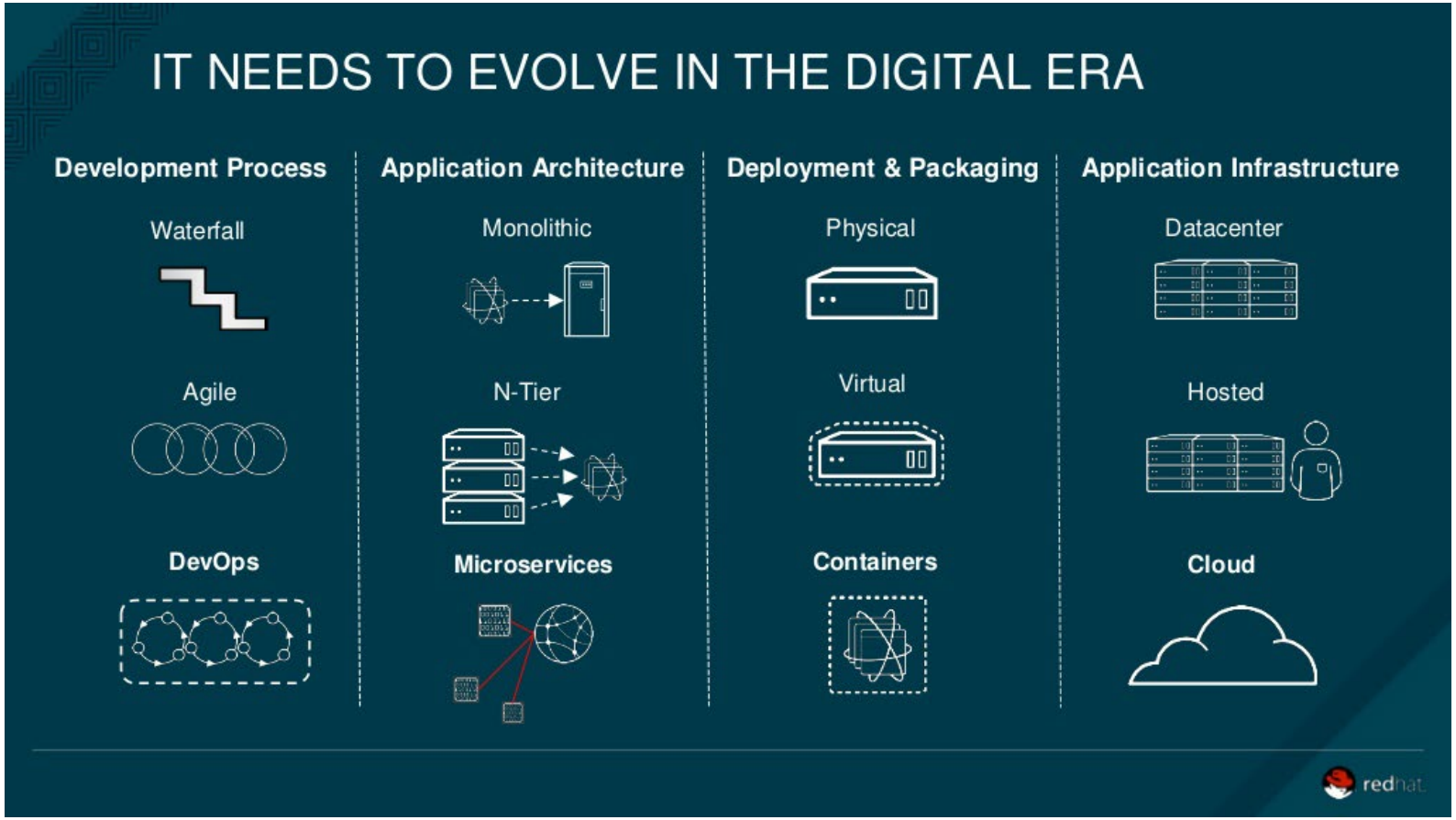
Var är lämpligast att drifta en ny lärplattform för högskolan?

- In-house?
- Co-location, hosting, outsourcing?
- Moln?
  - Privat moln?
  - Publikt moln?
    - SaaS?
    - IaaS?
- Hybrid?

Valet…

- …är en fråga om **skalekonomi**

Evolution för infrastruktur



Vad ska vi flytta till molnet?

- Standardfunktionalitet är enklast att flytta
  - Email
  - Office
  - Fillagring
  - Active Directory
  - Det som redan ska nås publikt
- System med höga sekretesskrav är svårast att flytta
  - Interna fakturasystem
  - Regionens journalsystem
  - Domstolsverkets målhanteringssystem

Hybrid-IT

- För ett par år sedan var det självklart för många att flytta till molnet på grund av skalekonomin, och minskad TCO
- Vissa tjänster fungerar väldigt bra för detta, särskilt webb-baserade tjänsterI de flesta organisationer är det dock svårt att flytta allt på grund av:
  - Lagstiftning och praxis (GDPR, NIS2 och MSB:s riktlinjer)
  - Saknas stöd i mjukvaran (legacy)
  - Kompetensen saknas på flera nivåer, motstånd i ledning och/eller IT-organisation
- En utav de vanligaste modellerna för drift i dagsläget är därmed hybrid. I många fall innebär hybrid-IT att man har:
  - Kvar stora delar av sin miljö som är on-premises
  - Flyttat upp, och flyttar upp, delar av miljön till publika molntjänster

Uppgift 3

kahoot.it

JÖNKÖPING UNIVERSITY

School of Engineering